```
  <b:project xmlns:b="http://some.domain/book_project">
    <b:book></b:book>
  </b:project>
</data>
```

The URI of the namespace is not that important; it doesn't need to be a valid URL, it just needs to be unique, though they are often used to point to pages containing information about the namespace. Another way to introduce namespaces into XML documents is to put them all in the root element like this…

```
<data xmlns:c="http://some.domain/code_project"
xmlns:b="http://some.domain/book_project">
  <c:project>
    <c:codebase></c:codebase>
  </c:project>
  <b:project>
    <b:book></b:book>
  </b:project>
</data>
```

… and yet another way to do it is to define a *default namespace* for the parent element and skip having to shove those prefixes in all the child elements:

```
  <project xmlns="http://some.domain/code_project">
    <codebase></codebase>
  </project>
```

It is this default namespace mechanism that contributed to my decision to demonstrate XPath with `Dom\XMLDocument`, because `Dom\HTMLDocument` automatically puts HTML that it parses into the default namespace of "http://www.w3.org/1999/xhtml" (note that `DOMDocument` does not exhibit this behavior). This is entirely correct behavior, but it would have meant dumping namespaces on you before you had a chance to get comfortable with the basics of HTML, XML, and DOM. A quick demo will show you what I'm talking about:

```
// ... $html5 heredoc as before
$dom = Dom\HTMLDocument::createFromString($html5);
$xpath = new Dom\XPath($dom);
```